

hashmap →

~~search~~ get | Contains key $O(1)$

① set/get $\Rightarrow$ Array | List Stack
↓ (queue)
Tree

key-value
↳ value

L value

A 88

 $\overline{f(n)}$

key	value
A	88
C	57 ✓
d	72 ✓
b	85 ✓

index

↓

Add 0(1)

$O(n)$ { get
remove
contains

bin / darf

A → 88
C → 57
D → 72
B → 85

Deckfun $\rightarrow$

$\hookrightarrow ASI: y =$
 $65\% \cdot y =$
 $66\% \cdot y =$
 $68\% \cdot y =$
 $69\% \cdot y =$

A → 85
C → 76
D → 57
B → 81
E → 73
F → 69

← LF → TRF →

$$LF = \frac{\text{total number of elements}}{\text{size}} = \frac{12}{4} = 3.0$$

$$LF = \frac{12}{8} = 1.5$$

hash fun

```

class Node {
    String key;
    int value;
    Node next;
}

```

```
public void put(K key, V value) {
    int i = hashfun(key); ✓
    Node temp = ll.get(i); ✓
    while (temp != null) {
        if (temp.key.equals(key))
            temp.value = value;
        return;
    }
    temp = temp.next;
}
Node nn = new Node();
nn.key=key;
nn.value=value;
temp = ll.get(i); ✓
nn.next = temp
```

Hand-drawn diagram of a Huffman tree. The root node is a rectangle divided into four sections: '12%', '45%', '6%', and '8%'. The '45%' section is crossed out with a large 'X'. To the right of the root, 'm, 87' is written and underlined. Below the root, there are four nodes: 'D, 52', 's, 4', 'B, 82', and 'C, 176'. 'D, 52' branches into 'H' and 'L'. 's, 4' branches into 'A, 85' and 'E, 176'. 'B, 82' branches into 'F, 63' and 'J'. 'C, 176' branches into 'G' and 'K'. 'A, 85' branches into 'I' and 'J'. 'F, 63' branches into 'J' and 'J'. 'E, 176' branches into 'J' and 'J'. 'J' branches into 'J' and 'J'.

Diagram illustrating the merging of two sorted arrays [10, 2] and [20, 3] into a new array [30, 4].

The process shows the following steps:

- Initial arrays: [10, 2] (labeled 1K) and [20, 3] (labeled 2K).
- Merge step: The elements are compared and merged into a new array [30, 4] (labeled 3K).
- Final result: The merged array [30, 4] is shown, with the element 4 circled and labeled 4K.

Below the main diagram, a sequence of operations is shown:

10 | 2K → 20 | 3K → 30 | 4K → 40 | 5K → 50 | 6K → 60 | 7K

The final result is 60 | 7K.

```
public V remove(K key) {
    int i = hashfun(key);
    Node curr = ll.get(i);
    Node prev = null;
    while (curr != null) {
        if (curr.key.equals(key)) {
            break;
        }
        prev = curr;
        curr = curr.next;
    }
}
```

no

```
private void reshaping() {
    // TODO Auto-generated method stub
    ArrayList<Node> new_ll = new ArrayList<>()
    for (int i = 0; i < 2 * ll.size(); i++) {
        new_ll.add(null);
    }
    ArrayList<Node> old_ll = ll;
    ll = new_ll;
    size = 0;
}
```

Diagram illustrating the reshaping process. A linked list structure is shown with 8 empty nodes (indices 0 to 7). The pointer `ll` points to the first node (index 0). The code below shows the reshaping logic:

```

private void reshaping() {
    // TODO Auto-generated method stub
    ArrayList<Node> new_ll = new ArrayList<>()
    for (int i = 0; i < 2 * ll.size(); i++) {
        new_ll.add(null);
    }
    ArrayList<Node> old_ll = ll;
    ll = new_ll;
    size = 0;
}

```

Handwritten notes below the diagram:

- for C Node nn: old_ll
- while (nn != null)
- put c nn-key,
- 3 nn->nn = next;

Given an array of strings `strs`, group the **anagrams** together. You can return the answer in **any order**.

— • —

Input: strs = ["eat","tea","tan","ate","nat","bat"]

Output: `[["bat"], ["pat", "tan"], ["ate", "eat", "tea"]]`

Explanation:

```

public static List<List<String>> group_Anagrams(String[] arr) {
    HashMap<String, List<String>> map = new HashMap<>();
    for (int i = 0; i < arr.length; i++) {
        String key = getKey(arr[i]);
        if (!map.containsKey(key)) {
            map.put(key, new ArrayList<>());
        }
        map.get(key).add(arr[i]);
    }
}

```

w	2
4	3
2	8

Given an array of strings `strs`, group the **anagrams** together. You can return the answer in **any order**.

Example 1.

Example 1:

Input: strs = ["eat", "tea", "tan", "ate", "nat", "bat"]

Output: `[["bat"], ["nat", "tan"], ["ate", "eat", "tea"]]`

Explanation

Explanation:

eat

100010---1---

← tea

a b c d e f --- T u v --- x y z

010100 ✓

"bbbbbbbbbddd", "bbbbbbbbbcb"

010100

010100

1
abcde

→ 010100

a b c d e

a b c d e